



Accélérateur de Talents Tech

EXERCICES

Formation Dev Web Full Stack



TypeScript

Exercices Pratiques

EXO 01

Côté Client

TypeScript + DOM

Prépare à React / .tsx

EXO 02

Côté Serveur

TypeScript + API REST

Node.js · Express · Prisma

worketyamo.com · +237 674 197 190

EXERCICE 01 Frontend · Client-side · Prépare à React

TaskBoard UI

Application de gestion de tâches côté client — TypeScript + DOM + Tailwind CSS

Contexte & Situation

Situation Problème

Tu rejoins l'équipe frontend de StartupCM, une startup camerounaise qui développe un outil de gestion de projets pour les équipes tech d'Afrique centrale.

Le designer a livré les maquettes Figma d'un tableau de tâches (TaskBoard). Le backend Node.js/Express est déjà en production et expose une API REST. Ton rôle : construire l'interface client en TypeScript pur — avant que l'équipe migre vers React dans le prochain sprint.

Le tech lead a posé une contrainte claire : "Zéro any dans le code livré. TypeScript strict mode. Si tu maîtrises ça ici, le passage à React .tsx sera naturel."

Ce que tu sais déjà

Module	Acquis
JavaScript ES6+	Variables, fonctions, classes, async/await, fetch, DOM, événements
TypeScript	Types de base, interfaces, génériques, classes, utility types, modules
HTML / CSS	Structure sémantique, formulaires, attributs data-*
Tailwind CSS	Classes utilitaires, dark mode, responsive, composants
Git / GitHub	Branches, commits conventionnels, PR

Partie A — Niveau Débutant

Objectif

Mettre en place la structure TypeScript du projet, typer toutes les données et afficher une liste de tâches statiques en manipulant le DOM avec TypeScript strict.

A1 — Setup & Configuration

Initialise un projet TypeScript from scratch avec la structure suivante :

```
taskboard/
├─ src/
│  └─ types/
```

```

├── index.ts          ← Toutes les interfaces et types
├── utils/
│   ├── dom.ts       ← Helpers DOM typés
│   ├── state/
│   │   ├── store.ts ← État global typé
│   │   └── components/
│   │       ├── TaskCard.ts ← Fonction de rendu d'une carte
│   │       └── TaskForm.ts ← Logique du formulaire
│   └── main.ts       ← Point d'entrée
├── index.html
├── tsconfig.json     ← strict: true obligatoire
└── package.json

```

A2 — Modélisation des données

Dans `src/types/index.ts`, déclare les types suivants :

- Un type `Statut` avec les valeurs littérales : `'TODO' | 'EN_COURS' | 'TERMINE' | 'ANNULE'`
- Un type `Priorite` avec : `'BASSE' | 'NORMALE' | 'HAUTE' | 'URGENTE'`
- Une interface `Tache` avec les propriétés : `id` (string), `titre` (string), `description` (string optionnelle), `statut` (Statut), `priorite` (Priorite), `creeLe` (Date), `termLe` (Date | null), `tags` (string[])
- Un type `CreerTacheDTO` en utilisant `Omit<>` pour exclure `id` et `creeLe` de `Tache`
- Un type `MettreAJourTacheDTO` en utilisant `Partial<CreerTacheDTO>`
- Une interface `EtatApp` avec : `taches` (Tache[]), `filtreStatut` (Statut | 'TOUTES'), `filtrePriorite` (Priorite | 'TOUTES'), `recherche` (string)

A3 — Données initiales

Dans `src/state/store.ts`, crée un tableau de 6 tâches fictives typées `Tache[]` couvrant les 4 statuts. Chaque tâche doit avoir au moins 2 tags. Exporte un état initial de type `EtatApp`.

Contrainte TypeScript A3

Utilise `crypto.randomUUID()` pour les ids. Typescript doit inférer le type de chaque propriété sans que tu aies à annoter chaque ligne — mais l'objet complet doit être annoté : `const taches: Tache[] = [...]`

A4 — Affichage DOM typé

Dans `src/components/TaskCard.ts`, crée une fonction :

```

function creerCarteDOM(tache: Tache): HTMLLIElement {
  // Retourne un <li> structuré avec Tailwind CSS
  // La carte doit afficher : titre, statut (badge coloré),
  // priorité (badge), tags (chips), date de création
  // Utiliser setAttribute('data-id', tache.id) pour les interactions futures
}

```

Dans `src/main.ts`, sélectionne l'élément `#liste-taches` et rends toutes les tâches en appelant `creerCarteDOM`. Utilise `querySelectorAll` avec le bon type `HTMLCollection`.

Contraintes TypeScript A4

- `document.querySelector('#liste-taches')` doit être casté avec `as HTMLUListElement` — jamais `any`
- Tous les éléments créés avec `createElement` doivent être typés : `HTMLLIElement`, `HTMLSpanElement`...
- La fonction `creerCarteDOM` doit avoir un type de retour explicite : `HTMLLIElement`

Partie B — Niveau Intermédiaire

Objectif

Ajouter les interactions, la logique de filtrage typée, la persistance `localStorage`, et structurer le code de façon à ce qu'il soit facilement migrable vers React.

B1 — Ajout de tâche (formulaire typé)

Dans `src/components/TaskForm.ts`, implémentes la logique du formulaire d'ajout :

- Écouter le submit du formulaire avec `e.preventDefault()`
- Lire les valeurs des champs et les typer explicitement (`HTMLInputElement`, `HTMLSelectElement`)
- Construire un objet de type `CreerTacheDTO` à partir des valeurs du formulaire
- Créer la Tache complète avec un id (`crypto.randomUUID()`) et `creeLe (new Date())`
- Ajouter la tâche à l'état (`EtatApp`) et re-rendre la liste
- Vider les champs et remettre le focus sur le champ titre

Validation côté TypeScript

Crée une fonction `validerDTO(dto: CreerTacheDTO): string[]` qui retourne un tableau d'erreurs (vide = valide). Si le titre est vide ou `< 3` caractères : erreur.

Si la priorité n'est pas dans les valeurs autorisées : erreur. Utilise `includes()` sur un tableau de type `readonly Priorite[]` pour la vérification.

B2 — Filtres et recherche typés

Implémente dans `src/state/store.ts` une fonction pure :

```
function filtrerTaches(  
  taches: Tache[],  
  filtre: Pick<EtatApp, 'filtreStatut' | 'filtrePriorite' | 'recherche'>  
) : Tache[] {  
  // Retourner les tâches filtrées — fonction pure, sans effets de bord  
}
```

- Chaque changement de filtre (événement `change/input`) doit appeler `filtrerTaches` et re-rendre

- Le compteur par statut doit se mettre à jour avec un `Record<Statut, number>` calculé via `reduce()`
- La recherche est case-insensitive sur titre et description

B3 — Changement de statut

Utilise la délégation d'événements sur `#liste-taches`. Au clic sur le badge de statut d'une carte :

- Récupère l'id depuis `dataset.id` (type `string`, puis caste en ce dont tu as besoin)
- Fais cycler le statut : `TODO` → `EN_COURS` → `TERMINE` → `TODO`
- Déclare le cycle comme un tuple readonly : `const CYCLE_STATUT = ['TODO', 'EN_COURS', 'TERMINE'] as const`
- Mets à jour la tâche dans l'état de façon immuable : `map()` + spread operator

B4 — Persistance `localStorage`

Dans `src/utlis/dom.ts`, crée un module de storage typé :

```
const storage = {
  sauvegarder<T>(cle: string, valeur: T): void {
    // JSON.stringify + localStorage.setItem
  },
  charger<T>(cle: string): T | null {
    // localStorage.getItem + JSON.parse + gestion d'erreur
  },
}
```

- Sauvegarder l'état complet `EtatApp` à chaque modification
- Au chargement de la page, restaurer l'état depuis `localStorage`
- Les `Date` sont sérialisées en `string` — convertis-les en `Date` à la désérialisation

B5 — Préparation à React (le pont conceptuel)

Pourquoi cette étape est cruciale

React est du TypeScript qui appelle des fonctions pour produire de l'UI. Avant de passer à `.tsx`, entraîne-toi à penser en composants : une fonction = un morceau d'UI = des props en entrée.

Refactorise ton code pour que chaque fonction de rendu suive ce pattern :

```
// Pattern que tu retrouveras IDENTIQUE en React

// Côté client vanilla (ce que tu fais maintenant) :
interface TaskCardProps {
  tache: Tache
  onStatutChange: (id: string, statut: Statut) => void
  onSupprimer: (id: string) => void
}

function TaskCard(props: TaskCardProps): HTMLLIElement { ... }
```

```
// En React (sprint suivant), ce sera EXACTEMENT :
function TaskCard({ tache, onStatutChange, onSupprimer }: TaskCardProps) {
  return <li>...</li> // JSX à la place de createElement
}
```

- Toutes tes fonctions de rendu doivent accepter un objet Props typé avec une interface
- Les callbacks (handlers) doivent être passés en props, jamais capturés par closure
- L'état (EtatApp) ne doit vivre qu'au niveau le plus haut (main.ts = futur App.tsx)

Livrables attendus

	Livable	Description
	Projet complet	Dépôt Git avec tsconfig.json strict: true, package.json et src/ structuré
	src/types/index.ts	Tous les types et interfaces sans aucun any
	src/state/store.ts	EtatApp, filtrerTaches(), persistance localStorage
	src/components/	TaskCard.ts et TaskForm.ts avec interfaces Props
	index.html	HTML sémantique avec Tailwind CSS, dark mode toggle
	Validation	0 erreur tsc --noEmit, 0 any dans le code livré

Critères d'évaluation

Critère	Points	Description
Typage strict	25 pts	0 any, paramètres annotés, types de retour explicites
Interfaces & types	20 pts	Omit, Partial, Record, union littérales bien utilisés
Génériques	15 pts	storage<T>, filtrerTaches, au moins 1 fonction générique
Pattern Props	15 pts	Toutes les fonctions de rendu avec interface Props
Fonctionnalités	15 pts	CRUD, filtres, recherche, persistance fonctionnels
Qualité du code	10 pts	Modules ES6, nommage, structure claire

Ressources utiles

TypeScript Handbook → <https://www.typescriptlang.org/docs/handbook/intro.html>

TS Playground → <https://www.typescriptlang.org/play>

MDN DOM API → https://developer.mozilla.org/fr/docs/Web/API/Document_Object_Model

Tailwind Docs → <https://tailwindcss.com/docs>

EXERCICE 02 Backend · Server-side · API REST

TaskFlow API

API REST typée côté serveur — TypeScript + Node.js + Express + Prisma + MongoDB

Contexte & Situation

Situation Problème

L'application TaskBoard (Exercice 01) est en production côté client. Le CEO de StartupCM veut maintenant ouvrir la plateforme à plusieurs équipes avec des comptes utilisateurs, des projets partagés et un historique d'activité.

Tu rejoins le pôle backend. Le système existant est un prototype Node.js/JavaScript sans types. La première mission : réécrire l'API en TypeScript strict avec Prisma comme ORM et MongoDB comme base de données, en respectant les conventions REST.

Le CTO a une règle non négociable : "Chaque route Express est typée. Chaque handler connaît le type exact de req.body, req.params et la réponse qu'il renvoie."

Ce que tu sais déjà

Module	Acquis
Node.js	Modules CommonJS/ESM, fs, path, process.env, EventEmitter
Express	Routes, middlewares, req/res/next, Router, gestion d'erreurs
MongoDB	Collections, documents, CRUD, indexes, ObjectId
Prisma	Schema.prisma, migrations, Prisma Client, relations, queries
TypeScript	Types, interfaces, génériques, utility types, classes, modules

Partie A — Niveau Débutant

Objectif

Mettre en place le projet TypeScript/Express, configurer Prisma avec MongoDB, et implémenter les types de l'API ainsi que les routes CRUD basiques des tâches.

A1 — Setup du projet

Initialise un projet Node.js/TypeScript avec la structure suivante :

```
taskflow-api/  
├─ src/
```

```

├── types/
│   ├── index.ts          ← Types globaux, DTOs
│   └── express.d.ts      ← Augmentation du type Request
├── middlewares/
│   ├── auth.middleware.ts
│   ├── validate.middleware.ts
│   └── error.middleware.ts
├── routes/
│   ├── taches.routes.ts
│   ├── projets.routes.ts
│   └── auth.routes.ts
├── controllers/
│   ├── taches.controller.ts
│   ├── projets.controller.ts
│   └── auth.controller.ts
├── services/
│   ├── taches.service.ts
│   └── auth.service.ts
├── app.ts                ← Express app + middlewares
├── prisma/
│   └── schema.prisma
├── .env
└── tsconfig.json        ← strict: true, module: CommonJS

```

Installe les dépendances nécessaires :

```

npm install express @prisma/client
npm install -D typescript ts-node @types/node @types/express prisma

```

A2 — Schéma Prisma (MongoDB)

Définis le schéma Prisma dans prisma/schema.prisma :

```

datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model Utilisateur {
  id          String  @id @default(auto()) @map("_id") @db.ObjectId
  email       String  @unique
  motDePasse String
  prenom      String
  nom         String
  role        Role    @default(USER)
  creeLe      DateTime @default(now())
  projets     ProjetMembre[]
  taches      Tache[] @relation("AssigneA")
}

model Projet {

```

```

    id          String          @id @default(auto()) @map("_id") @db.ObjectId
    nom          String
    description  String?
    creeLe      DateTime        @default(now())
    taches      Tache[]
    membres     ProjetMembre[]
  }

  model Tache {
    id          String          @id @default(auto()) @map("_id") @db.ObjectId
    titre       String
    description  String?
    statut      Statut          @default(TODO)
    priorite    Priorite        @default(NORMALE)
    tags        String[]
    creeLe      DateTime        @default(now())
    echeance    DateTime?
    projetId    String          @db.ObjectId
    projet      Projet          @relation(fields: [projetId], references: [id])
    assigneeId  String?         @db.ObjectId
    assignee    Utilisateur?    @relation("AssigneA", fields: [assigneeId], references:
[id])
  }

  model ProjetMembre {
    id          String          @id @default(auto()) @map("_id") @db.ObjectId
    utilisateur String          @db.ObjectId
    utilisateur Utilisateur     @relation(fields: [utilisateurId], references: [id])
    projetId    String          @db.ObjectId
    projet      Projet          @relation(fields: [projetId], references: [id])
    roleProjet  RoleProjet      @default(MEMBRE)
  }

  enum Statut   { TODO EN_COURS TERMINE ANNULE }
  enum Priorite { BASSE NORMALE HAUTE URGENTE }
  enum Role     { ADMIN USER }
  enum RoleProjet { OWNER ADMIN MEMBRE }

```

A3 — Types & DTOs

Dans `src/types/index.ts`, définis les types TypeScript qui correspondent aux modèles Prisma :

- Importe et ré-exporte les types Prisma générés : `import type { Tache, Utilisateur, Projet } from '@prisma/client'`
- Crée un type `CreerTacheDTO` avec : `titre` (string), `description` (string optionnel), `priorite` (Priorite optionnel), `projetId` (string), `assigneeId` (string optionnel), `tags` (string[] optionnel), `echeance` (Date | string optionnel)
- Crée `MettreAJourTacheDTO` = `Partial<Omit<CreerTacheDTO, 'projetId'>>`
- Crée un type `TacheAvecRelations` = `Tache & { projet: Projet; assignee: Utilisateur | null }` en utilisant les types Prisma
- Crée une interface `PaginationQuery` : `page` (number), `limit` (number), `tri` (string optionnel), `ordre` ('asc' | 'desc' optionnel)
- Crée un generic `ApiReponse<T>` : `{ success: boolean; data: T; message: string; meta?: PaginationMeta }`
- Crée `PaginationMeta` : `{ total: number; page: number; limit: number; totalPages: number }`

A4 — Routes CRUD /api/taches

Dans `src/routes/taches.routes.ts` et `src/controllers/taches.controller.ts`, implémente les routes suivantes avec Express et Prisma — toutes typées :

Méthode	Route	Action	Auth
GET	/api/taches	Liste paginée, filtrable par statut, priorité, projectId	Oui
GET	/api/taches/:id	Détail d'une tâche avec ses relations (projet, assignee)	Oui
POST	/api/taches	Créer une tâche — body : <code>CreerTacheDTO</code>	Oui
PATCH	/api/taches/:id	Mise à jour partielle — body : <code>MettreAJourTacheDTO</code>	Oui
DELETE	/api/taches/:id	Supprimer (soft delete via statut ANNULE)	Oui
PATCH	/api/taches/:id/statut	Changer uniquement le statut	Oui

Typage des handlers Express

Chaque controller doit utiliser les types Express génériques :

```
import { Request, Response, NextFunction } from 'express'

type Handler<Params = {}, ResBody = unknown, ReqBody = unknown, Query = {}> =
  (req: Request<Params, ResBody, ReqBody, Query>, res: Response<ApiReponse<ResBody>>,
    next: NextFunction) => Promise<void>

export const getTaches: Handler<{}, TacheAvecRelations[], {}, TachesQuery> = async (req, res) =>
{ ... }
```

Partie B — Niveau Intermédiaire

Objectif

Ajouter l'authentification JWT typée, un système de middleware de validation générique, la gestion d'erreurs centralisée typée, et les routes Projets avec contrôle d'accès.

B1 — Authentification JWT typée

Installe `jsonwebtoken` et `bcryptjs` avec leurs types :

```
npm install jsonwebtoken bcryptjs
npm install -D @types/jsonwebtoken @types/bcryptjs
```

Dans `src/types/express.d.ts`, augmente le type `Request` pour y ajouter l'utilisateur connecté :

```
// Augmentation de module — ajoute 'utilisateur' à tous les Request Express
import { Utilisateur } from '@prisma/client'

declare global {
  namespace Express {
    interface Request {
      utilisateur?: Omit<Utilisateur, 'motDePasse'>
    }
  }
}
```

Dans `src/middlewares/auth.middleware.ts` :

- Lire le token depuis l'en-tête `Authorization: Bearer <token>`
- Vérifier le token avec `jwt.verify()` — annoter le payload avec une interface `JwtPayload` typée
- Charger l'utilisateur depuis Prisma et l'attacher à `req.utilisateur`
- Retourner 401 si token absent ou invalide — type de l'erreur renvoyée : `ApiReponse<never>`

B2 — Middleware de validation générique

Crée un middleware générique de validation dans `src/middlewares/validate.middleware.ts` :

```
// Utilise Zod ou une validation manuelle typée
// Signature attendue :

type SchemaValideur<T> = {
  valider(donnees: unknown): { succes: true; data: T } | { succes: false; erreurs: string[] }
}

function validerBody<T>(schema: SchemaValideur<T>) {
  return (req: Request, res: Response, next: NextFunction): void => {
    const resultat = schema.valider(req.body)
    if (!resultat.succes) {
      res.status(400).json({ success: false, data: null, message: 'Validation échouée',
        erreurs: resultat.erreurs })
      return
    }
    req.body = resultat.data // Le body est maintenant typé T
    next()
  }
}
```

Implémente un validateur pour `CreerTacheDTO` (sans dépendance externe — fonctions pures TypeScript). Applique ce middleware sur la route `POST /api/taches`.

B3 — Gestion d'erreurs centralisée

Crée une hiérarchie d'erreurs métier dans `src/types/index.ts` :

```
class ErreurApp extends Error {
```

```

    constructor(
      message: string,
      public readonly codeStatut: number,
      public readonly code: string,
    ) { super(message); this.name = 'ErreurApp' }
  }

  class ErreurValidation extends ErreurApp {
    constructor(public readonly champs: Record<string, string>) {
      super('Données invalides', 400, 'VALIDATION_ERROR')
      this.name = 'ErreurValidation'
    }
  }

  class ErreurNonTrouve extends ErreurApp {
    constructor(ressource: string) {
      super(`${ressource} introuvable`, 404, 'NOT_FOUND')
    }
  }

  class ErreurNonAutorise extends ErreurApp {
    constructor() { super('Action non autorisée', 403, 'FORBIDDEN') }
  }

```

Dans `src/middlewares/error.middleware.ts`, crée le middleware d'erreur centralisé (4 paramètres : `err`, `req`, `res`, `next`). Il doit gérer `ErreurApp`, les erreurs Prisma (`PrismaClientKnownRequestError`) et les erreurs inconnues de façon distincte, et toujours renvoyer une `ApiResponse<never>` structurée.

B4 — Routes Projets avec contrôle d'accès

Implémente les routes `/api/projets` avec contrôle de rôle. Un utilisateur ne peut accéder qu'aux projets dont il est membre. Le propriétaire peut ajouter/retirer des membres.

Méthode	Route	Accès requis
GET	<code>/api/projets</code>	Utilisateur authentifié — liste ses projets
POST	<code>/api/projets</code>	Utilisateur auth — devient OWNER automatiquement
GET	<code>/api/projets/:id</code>	Membre du projet uniquement
GET	<code>/api/projets/:id/taches</code>	Membre du projet — tâches paginées
POST	<code>/api/projets/:id/membres</code>	OWNER ou ADMIN du projet
DELETE	<code>/api/projets/:id/membres/:uid</code>	OWNER du projet uniquement

Service de contrôle d'accès typé

Crée une fonction dans `src/services/acces.service.ts` :

```

async function verifierAcces(
  utilisateurId: string,
  projetId: string,
  roleRequis: RoleProjet[]

```

```
): Promise<ProjetMembre>
```

Elle lève `ErreurNonTrouve` si le projet n'existe pas, `ErreurNonAutorise` si le rôle est insuffisant. Retourne le `ProjetMembre` si l'accès est accordé.

B5 — Service d'historique (bonus)

Bonus optionnel

Crée un modèle `HistoriqueActivite` dans Prisma et un service qui enregistre chaque action (création, mise à jour, changement de statut) avec un type discriminé :

```
type TypeActivite = 'TACHE_CREEE' | 'TACHE_MISE_A_JOUR' | 'STATUT_CHANGE' | 'MEMBRE_AJOUTE'
```

```
interface HistoriqueItem {
  type: TypeActivite
  utilisateurId: string
  projetId: string
  entiteId: string
  meta: Record<string, unknown>
  creeLe: Date
}
```

Livrables attendus

	Livrable	Description
	Projet complet	Dépôt Git, tsconfig.json strict: true, .env.example
	schema.prisma	Modèles complets avec relations et enums
	src/types/	DTOs, <code>ApiReponse<T></code> , hiérarchie d'erreurs, <code>express.d.ts</code>
	src/routes/ + controllers/	Toutes les routes CRUD typées
	Auth middleware	JWT vérifié, <code>req.utilisateur</code> typé
	validate + error middleware	Validation générique, erreurs centralisées
	Validation	0 erreur tsc --noEmit, 0 any non justifié

Critères d'évaluation

Critère	Points	Description
Typage strict	25 pts	0 any, handlers typés, Prisma types utilisés
Architecture	20 pts	Séparation routes/controllers/services, Express.d.ts

Sécurité & Auth	20 pts	JWT typé, middleware auth, contrôle d'accès par rôle
Validation & Erreurs	15 pts	Middleware générique, hiérarchie d'erreurs, codes HTTP
Prisma & Base de données	15 pts	Relations, queries typées, pas de raw queries inutiles
Qualité du code	5 pts	Nommage, modules, lisibilité

Ressources utiles

TypeScript Handbook → <https://www.typescriptlang.org/docs/handbook/intro.html>
Express + TypeScript → <https://expressjs.com/> + [@types/express](https://github.com/DefinitelyTyped/DefinitelyTyped/tree/master/types/express)
Prisma avec MongoDB → <https://www.prisma.io/docs/guides/database/mongodb>
Prisma Client TypeScript → <https://www.prisma.io/docs/concepts/components/prisma-client>
JWT (jsonwebtoken) → <https://github.com/auth0/node-jsonwebtoken#readme>
Zod (validation) → <https://zod.dev> (optionnel mais recommandé)

Bonne chance ! 🚀

Ces deux exercices constituent la passerelle vers React et l'architecture full stack.
Maîtriser le typage strict ici rendra chaque concept React/Next.js naturel et évident.

◇ Worketyamo · worketyamo.com · +237 674 197 190